Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



CATEGORY

Future of Work & Enterprise Productivity

PROJECT

NOX

Enterprise intelligence, orchestrated.
Every mission, delivered.

TEAM



AstroRangers

Mission Commander
TEAM LEADER



Akash Bajpai
SDE 3 · Walmart Global Tech

Flight Director

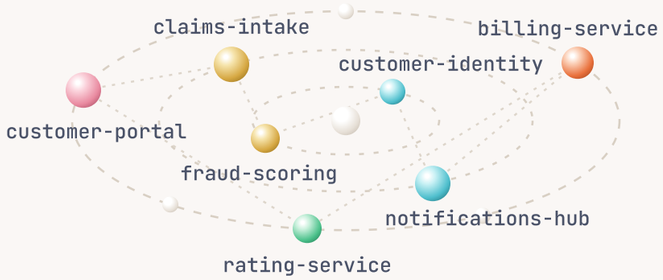


Kritika Singh
SDE · Lowe's India

NOX

In any

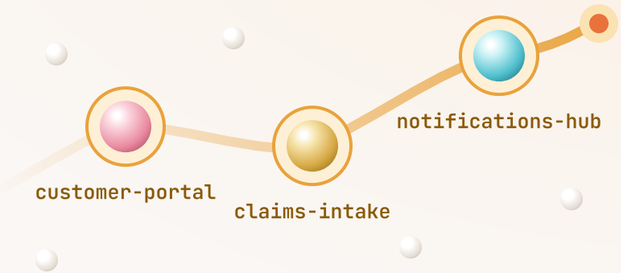
Enterprise



1

Hundreds of apps, dozens of teams.

All connected through APIs and events.



2

Every change crosses multiple apps.

One change touches many apps and teams.

JIRA · TW-2214
“Add photo upload to the claim form.”

SLACK · #TW-CLAIMS
“Max 10 photos, please.”

?

CONFLUENCE · POLICY
“Delete photos after 7 years.”

SOMEONE’S HEAD
“Fraud scoring reads every upload.”

3

Requirements are scattered.

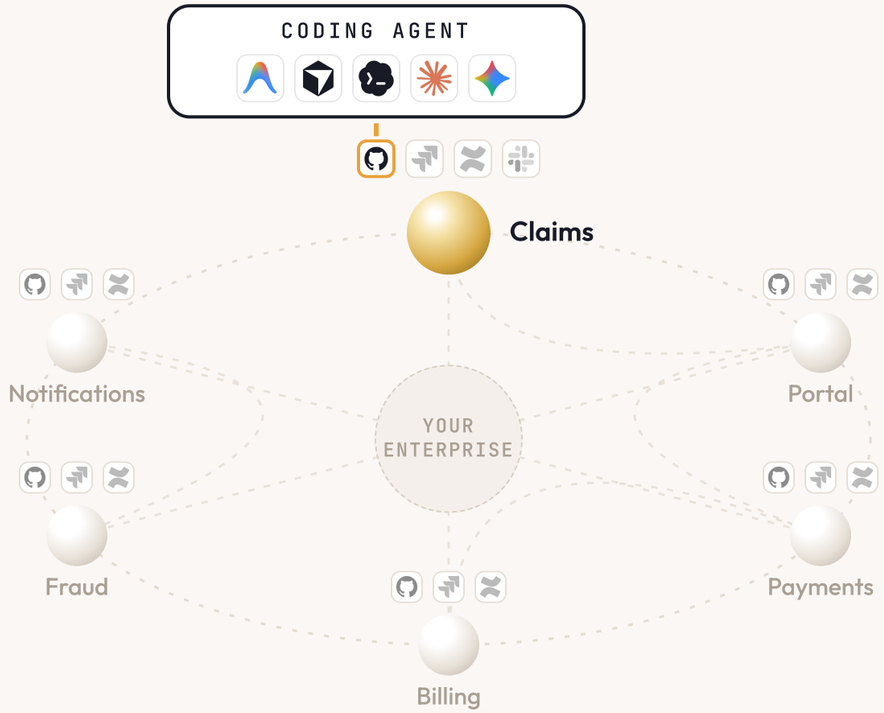
A ticket, a chat, a doc and someone’s head.

↓ And today, every change finally lands with **an AI coding agent** to build it.

Knowledge lives across many connected systems.

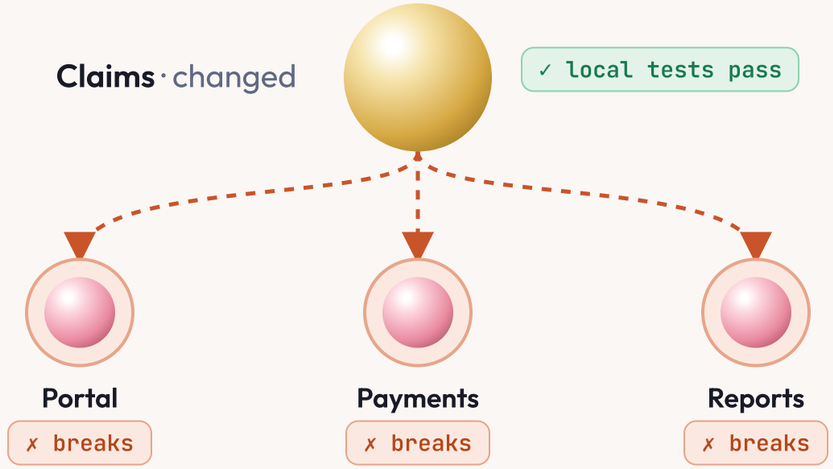
1 Coding agents see only one repo

They read the code. Docs, tickets, chats and other apps stay dark.



2 They can't see what a change breaks

Apps depend on each other through APIs and events. Change one, and every app that relies on it can fail. The agent only runs its own tests, so other teams find out in production.



THE GIST Coding agents see one repo, so fast code **quietly breaks other teams**.

The problem

NOX

A change's intent gets lost, and is never verified.

1 Answers get lost between people
Answers and decisions never reach the context the team builds from.

2 Tasks get checked, intent doesn't
Every task can be done while the original goal is still unmet.

BUSINESS ASKS
"Show customers who's handling their claim."

PRODUCT ASKS, IN A SLACK THREAD
"What if a team holds the claim, not one person?"
↳ "Then show the team name."

TICKET "Show the handler's name on the claim page."
`✗ the team answer never made it in`

✕ **Built:** team-held claims say "Not assigned yet". Those customers keep calling.

WHAT GETS CHECKED

- ✓ Tasks done
- ✓ Code merged
- ✓ Tests pass
- ✓ Ticket closed

WHAT NEVER GETS CHECKED

The original intent:
"Customers stop calling to ask where their claim is."

? **Was it met? Nobody verifies.**

✕ **The sprint closes. The calls don't drop.**

THE GIST Work gets done, but **not what was meant**, and nobody checks the goal at the end.

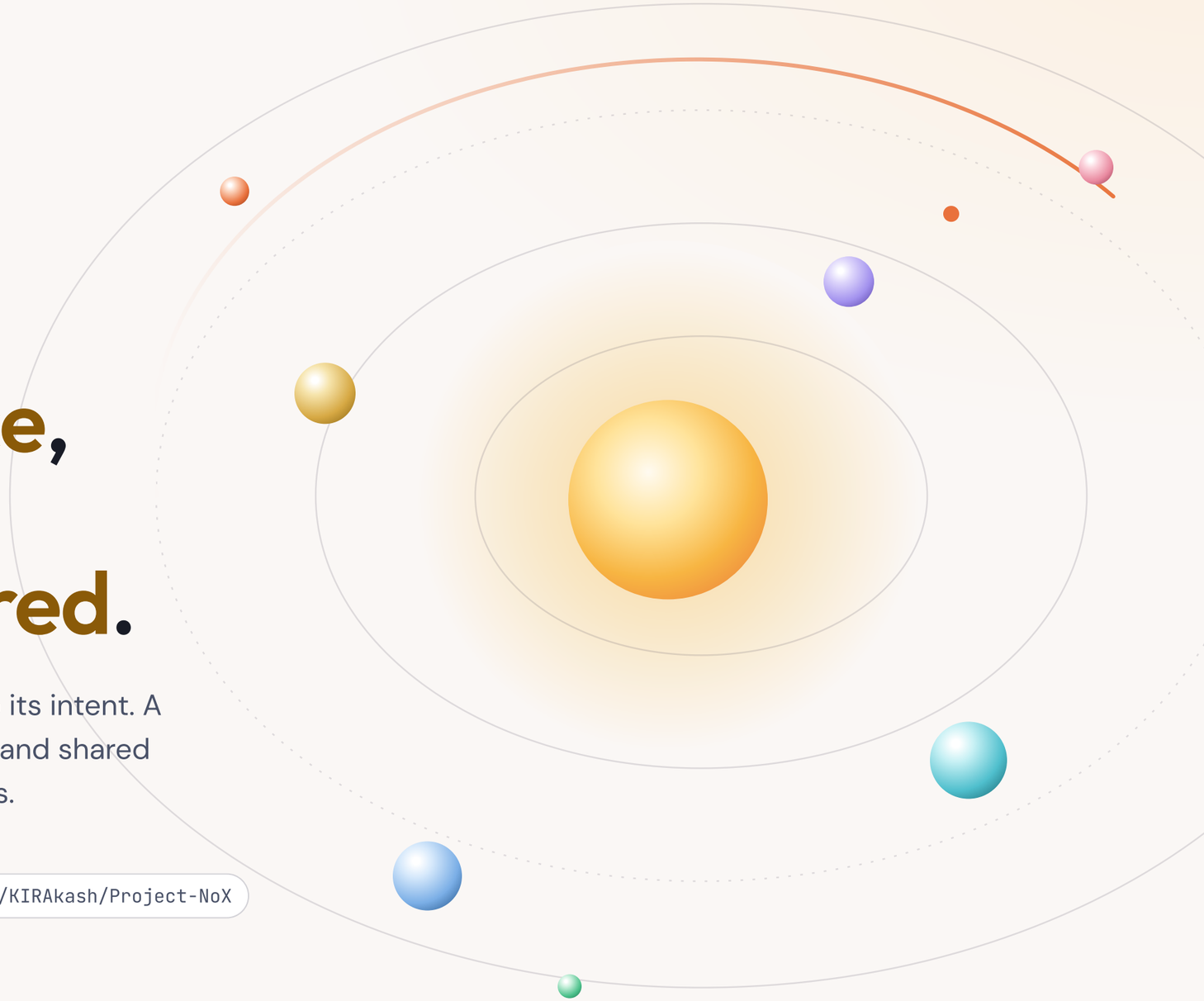
PRESENTING

NOX

Enterprise intelligence, orchestrated. Every mission, delivered.

From one sentence to verified code, with every spec held to its intent. A live map of the enterprise for every team and coding agent, and shared requirements written by every role and verified when it ships.

- Live app · **nox.example/app**
- 3-minute film · **link**
- github.com/KIRAkash/Project-NoX



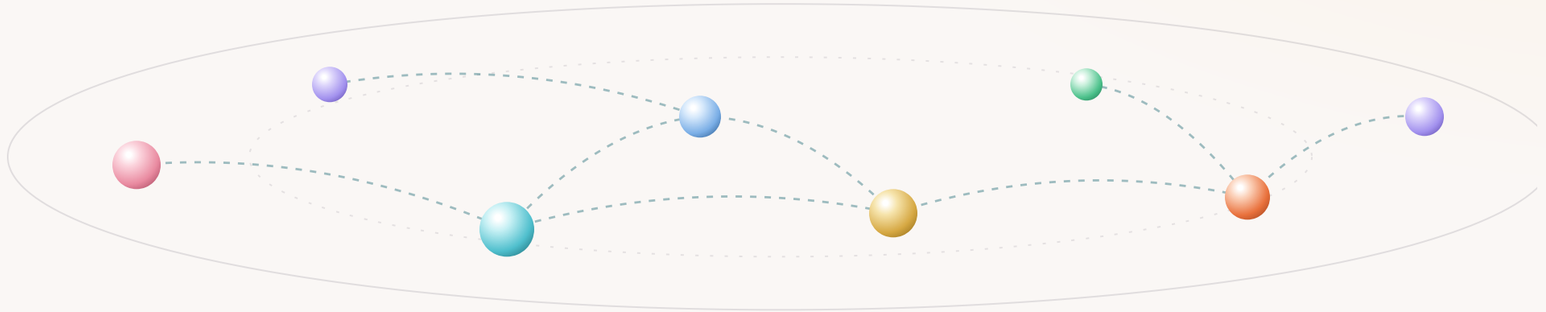
Meet NoX

NOX

Two verticals, one platform: know the enterprise, change it right.

NOX Atlas

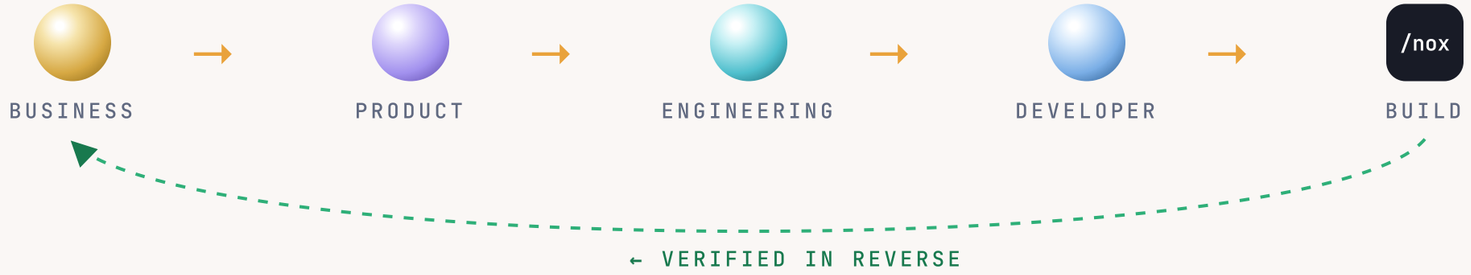
A live, cited map of every app, team and how they connect.



↓ the map feeds every mission

NOX Missions

One change, from shared requirements to shipped code, checked at the end.

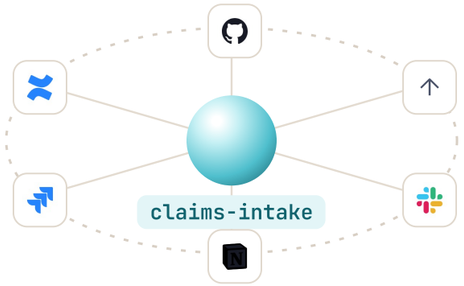


NOX Atlas

A living knowledge base for every application, **written by Gemini agents** and kept in Git, **reviewable by your team**.

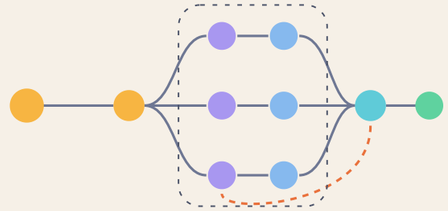
- Follows Google's **Open Knowledge Format**
- Built on the LLM-wiki pattern from **Andrej Karpathy**
- **Interlinked pages**, with **every app-to-app interaction** tracked
- Every page **cites its own sources**
- Searchable by meaning, with **Gemini embeddings**
- Can **run on-device with Gemma**, for private code
- **Optionally reviewed by a human** before it goes live
- **Updated on every change** to production

1 Every source of one app



2 NoX Atlas agents

Gemini agents, in series and in parallel.



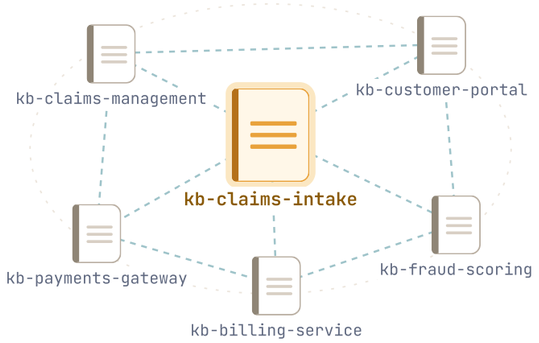
NoX Shield screens every source

3 A new Git repo for it

```
kb-claims-intake/
· AGENTS.md
· index.md
▸ concepts/
  · claim-intake-flow.md
▸ decisions/
  · adr-event-driven-intake.md
▸ entities/
  · claim.md
```

optional human review

4 Joins the knowledge map



On every change, **only the affected pages update**

NOX

NOX Atlas

Inside the build: a team of Gemini agents.

1 Ingest and screen

Sources, one app

Filtered snapshot
Noise removed
`Cloud Storage`

Diagrams kept
Read as images

NoX Shield
Flagged files withheld
`Model Armor`

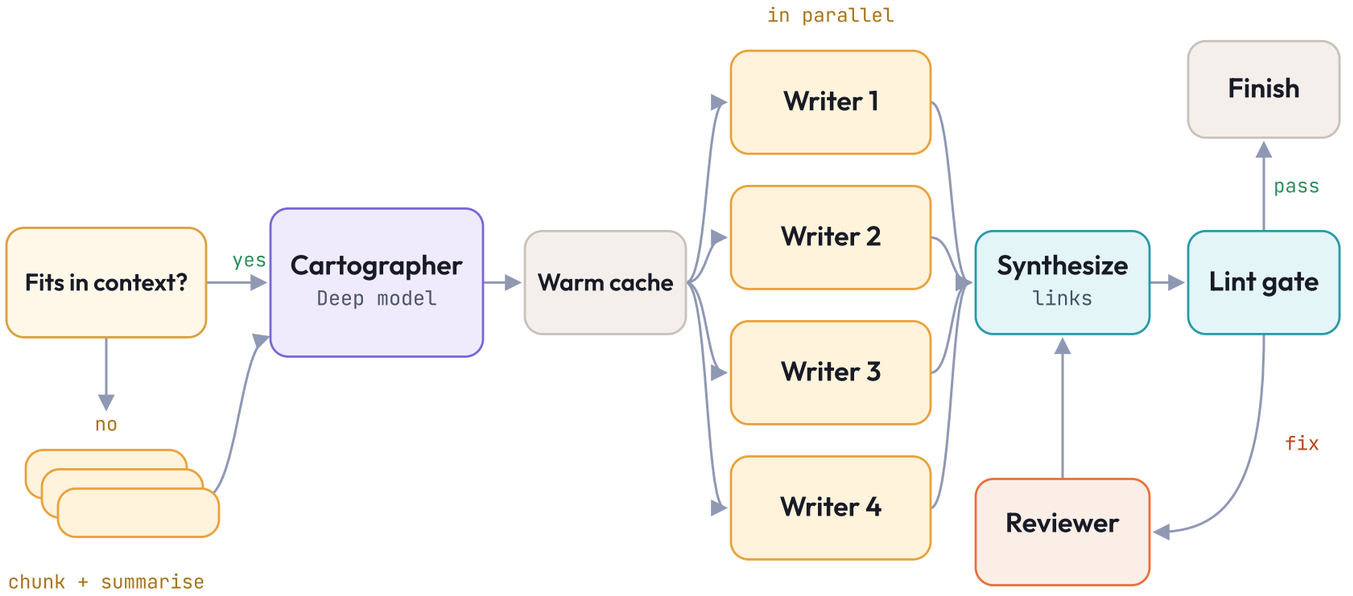
Contract discovery
APIs and events found

2 Google ADK workflow graph

LLM agent · deterministic node · loop

Every agent runs on **Gemini**

8.7× faster than one big model call



3 Check and publish

Contract map
Whole organisation

Search index
Changed chunks only
`gemini-embedding-2`

Secret gate
Secrets blocked
`Sensitive Data Protection`

New Git repo + PR
Opened as a PR
`kb-claims-intake`

Watch every source
Patches on every change

Gemini
Fast, Default and Deep tiers, long context, implicit caching

Google ADK 2
Workflow graph, LlmAgents, typed outputs

Agent Platform
Gemini by service account, fallback model

Model Armor + SDP
Screens what goes in and what is committed

Storage, SQL, Trace
Checkpoints, pgvector search, traces

Build on top of NoX Atlas, for your agents and for you.

For your agents

/nox skill
for six coding agents

★ 6 agents, one command

Loads the mission and its knowledge, then builds it.

```
$ nox init antigravity  →  /nox NOX-12
```

MCP server
10 read-only tools

Tools any MCP agent can call, scoped by the person's token.

`search_kb` `read_kb_page` `find_interfaces` `grep_source` `get_mission` `ask_nox` `+4`

```
$ nox mcp install all
```

A2A agent
built with Google ADK

Other enterprise agents find NoX and ask it questions, as a peer.

cited answers · same scope as you · NoX Shield on every question

```
GET /a2a/ask/.well-known/agent-card.json
```

NOX Atlas

Every app's knowledge base, one source for all six

Each knowledge-base repo also ships its own **AGENTS.md** and agent brief.

For you

Web app
Ask, explore, correct

Ask the knowledge base, explore its pages and fix what is wrong.

Ask, with citations · Listen and speak · Pin a correction · Contract map · Org knowledge base · Team lessons

```
Ask: who consumes claims.handler.assigned?
```

nox CLI
18 commands

Search, read and build knowledge bases from the terminal.

```
$ nox search "claim status"
$ nox read claims-management/entities/claim
$ nox context NOX-12  specs + pages
```

NoX Local
Gemma 4 · Ollama

★ Code never leaves the laptop

The same ADK agents, on the developer's laptop. Only finished pages are sent.

```
$ nox kb build --push  ·  nox kb watch
```

NOX

NOX Missions

One change, from a sentence to verified code.

1 Start a mission

What would you like to change?

1 Customers keep calling to ask where their claim is. Can they just see who's handling it?

Or just say it 2

3 SHOW NOX

Record screen | Take screenshot | Voice note | Upload

`recording · 0:42` `claim-page.png` `claims-faq.pdf`

4 ✦ `customer-portal` Send request →

1. **Type it** one sentence, in plain words
2. **Say it** Gemini writes it down, any language
3. **Show it** screen, screenshot, voice or a file. Gemini finds the page and the code
4. **NoX picks the app** from what you wrote and showed

2 Multiple roles write the spec, in order

`NoX co-writes every file`

BUSINESS	PRODUCT	ENGINEERING	DEVELOPER	/nox BUILD
`01-business.md`	`02-product.md`	`03-engineering.md`	`04-developer.md`	`nox/NOX-12`
The ask, why now, what done looks like	Stories, acceptance criteria, edge cases	Apps changing, contracts, rollout	Files, tasks, tests	Any coding agent, or Jules
✓ approved	✓ approved	✓ approved	✓ approved	

Then each seat verifies its own checklist, in reverse order

`NOX-12` business → product → engineering → developer → build → verifying → done

- **One person per file**
  Other seats can read it, not edit it.
- **NoX drafts, people approve**
  The next seat starts from a NoX draft.
- **Every line has a source**
  Unknowns become open questions.
- **Change upstream, see it**
  Files below an edit go stale.

3 Done

Done when **every requirement** in the spec is verified.

Item by item, with evidence: a PR, a CI run, a metric or a recording.

✗ `Failed? Send back`
NoX keeps the lesson.

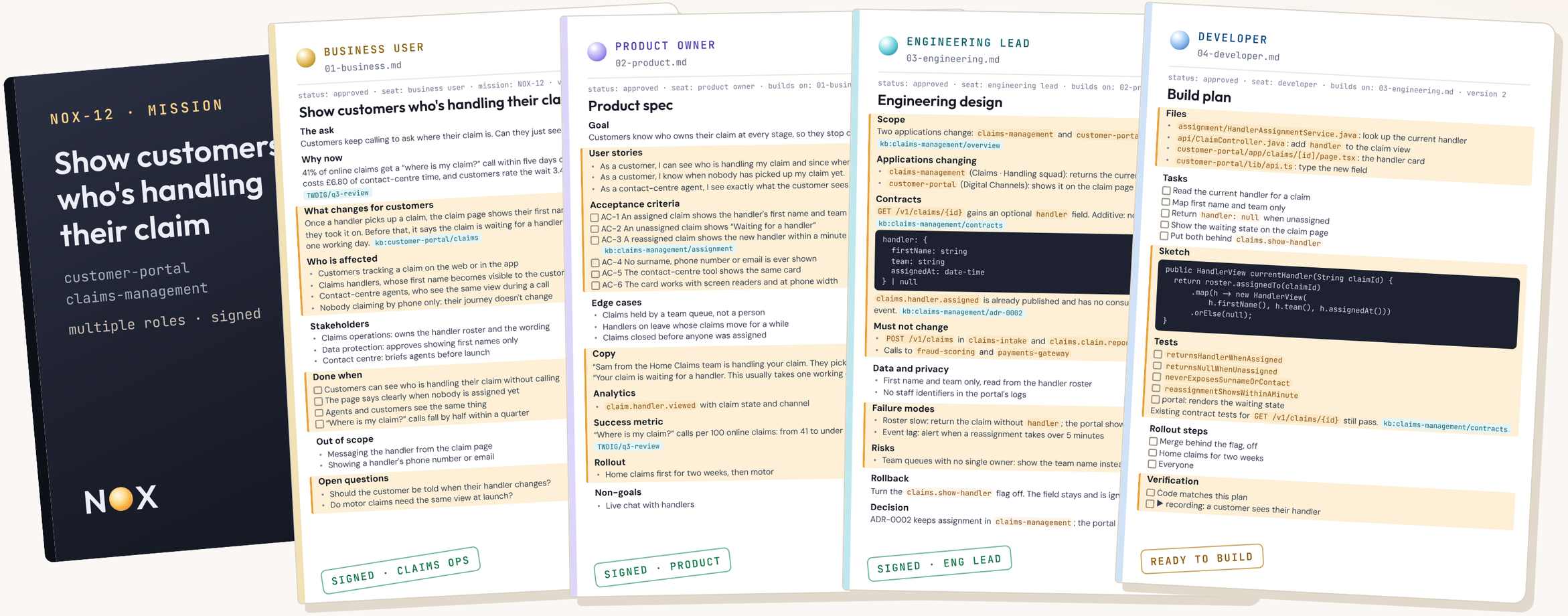
Every file kept in Git

Jira kept in step

The spec books

NOX

Every role writes its own detailed spec, with NoX.



written by the person drafted by NoX, with sources | Each page is signed and locked before the next role starts

One line in, a grounded spec out.

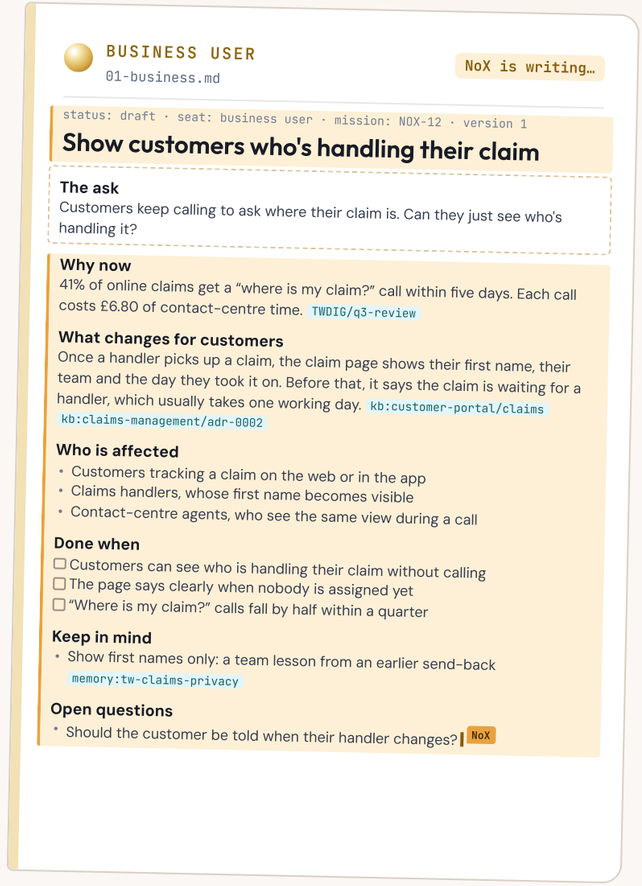
1 YOU WRITE

Customers keep calling to ask where their claim is. Can they just see who's handling it?

2 NOX READS THE ATLAS

search_kb	"claim handler" in your apps	✓
read_kb_page	customer-portal / claims	✓
find_interfaces	GET /v1/claims/{id}	✓
read_kb_page	claims-management / adr-0002	✓
source	TWDIG / q3-review	✓
team lessons	1 for this app and seat	✓

3 NOX WRITES, CITING EACH LINE



THEN KEEP WRITING TOGETHER

Refine

~~make sure ppl can see who has there claim~~

↓

Customers can see who is handling their claim.

Tidies what you wrote, keeps your meaning.

Chat

Add a section on data privacy

+ Data and privacy · 2 points added

Ask for a section or an edit. One click undoes it.

Context

▶ recording.mp4 ▣ screenshot.png ▤ claims-faq.pdf

3 facts added, each cited

Add evidence. NoX pulls facts from it.

The signed book goes to any coding agent.

Jules
Gemini CLI
Cursor
Codex
Copilot
Claude Code

NOX-12
Show customers who's handling their claim
4 signed pages
NOX

/nox

Google Antigravity — claims-management · branch nox/NOX-12

› /nox NOX-12

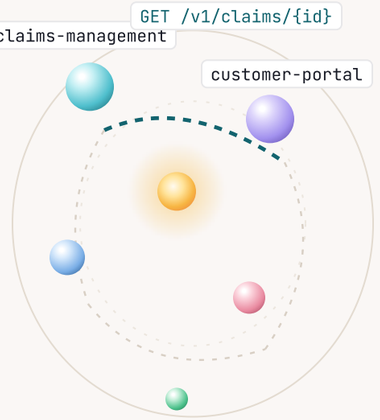
THE AGENT LOADS THE BOOK, THEN ASKS THE ATLAS

nox	get_mission	NOX-12	→ 4 signed pages
nox	search_kb	"claim view handler"	→ 3 pages
nox	read_kb_page	claims-management/contracts	→ read
nox	find_interfaces	GET /v1/claims/{id}	→ 1 consumer: customer-portal
nox	grep_source	HandlerAssignmentService	→ 2 matches

THEN BUILDS, GROUNDED

edit	HandlerAssignmentService.java	+18
edit	ClaimController.java	+6
edit	app/claims/[id]/page.tsx	+41
test	5 new tests pass	✓
nox pr	PR opened · guard checks the contracts	✓

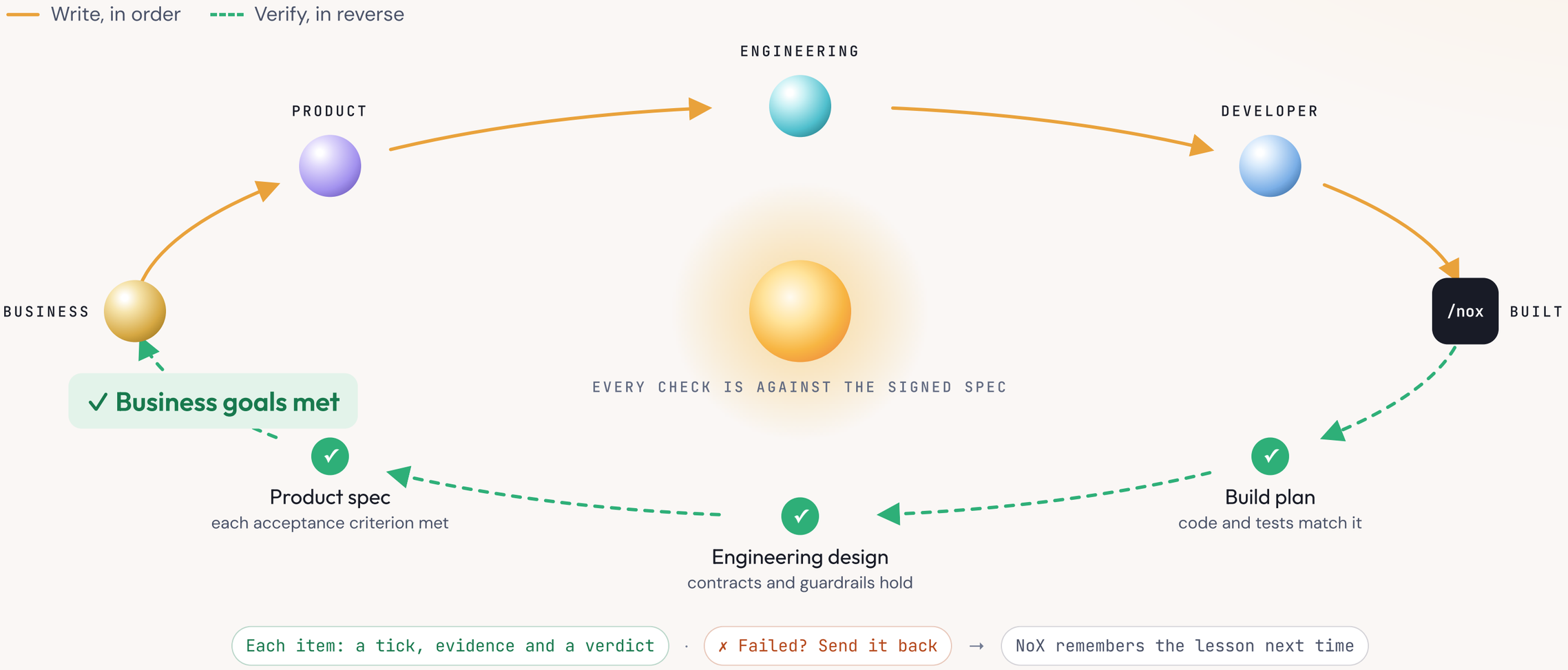
ATLAS · OVER MCP



Pages, contracts, code and team lessons, scoped to what this developer can see.

Same book, same Atlas, in every agent: `nox` CLI + MCP

Done means every requirement is verified.



Sightings

NOX Missions

NoX finds problems before anyone reports them.

Every week, a scout agent for each role reads the map and spots changes worth making.

Impact: High | Effort: Low

Customer experience

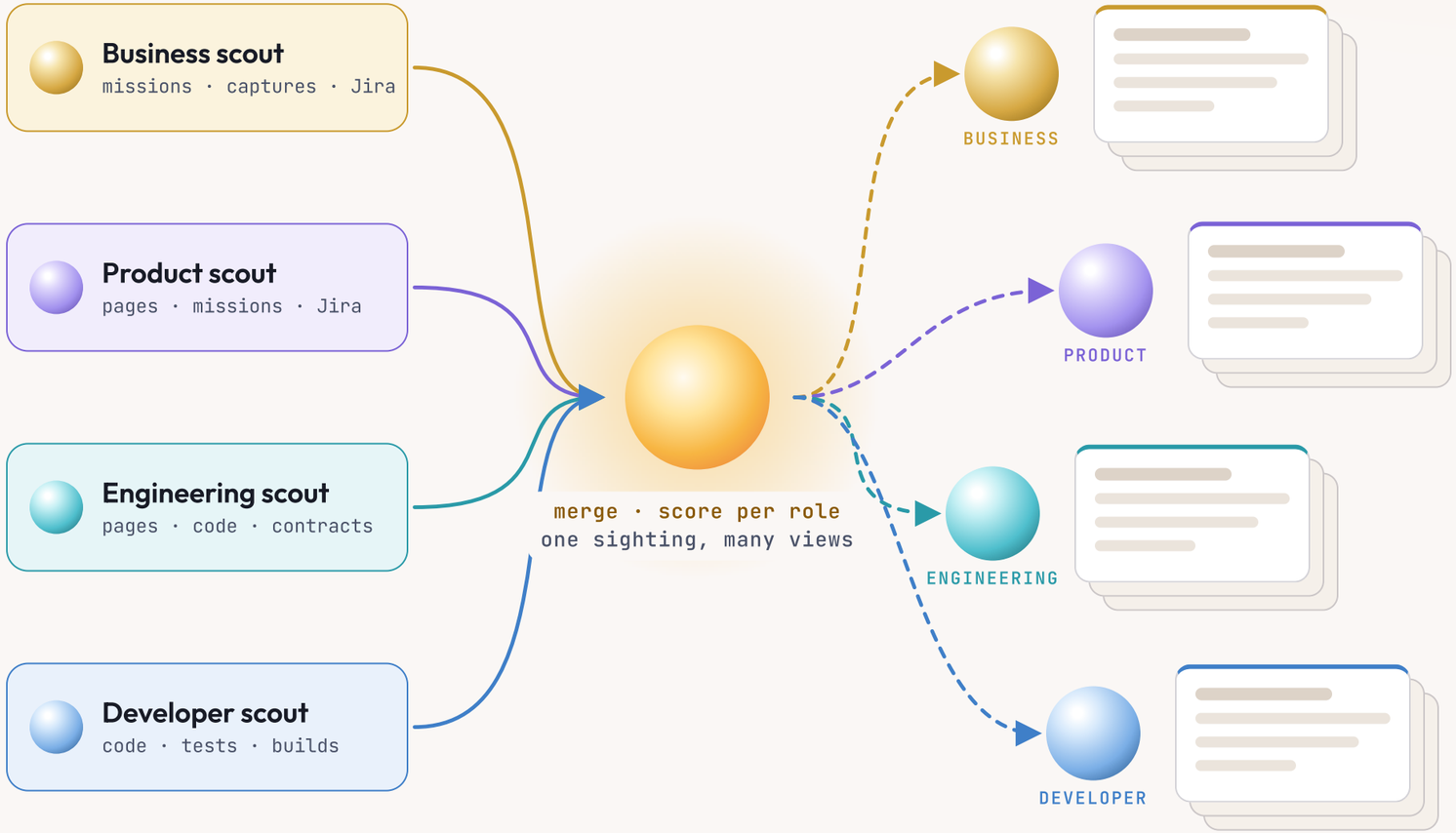
Show assigned claims handler details on the customer tracking page

Customers can't see if anyone has picked up their claim. **41%** of online claims lead to a contact-centre call within 5 days, at **£6.80** a call.

Fewer "where is my claim?" calls, and happier customers (3.4/5 today).

How NoX knows ⌄ ↳ TWDIG/digital-claims-review-q3

Start mission | Edit first | Not now | Dismiss



What changes when a large enterprise runs on NoX.

OPERATING GAP	TODAY	WITH NOX
Finding context	Meetings, Slack threads, asking around. A developer loses a day to “where does this live?”	Each seat starts from a cited brief drafted from the live map, in its own language.
Cross-team impact	Found in testing, or in production.	The blast radius sits on the design, before code: every consumer of a changed contract.
Hand-offs	Each person retells the request. Intent leaks at every hop.	The original sentence travels with the mission. Every page builds on the signed pages above it.
AI at work	Private chats. Coding agents start cold and can't see other teams.	One shared, scoped, cited context for every person and every agent, over MCP and A2A.
Definition of done	The ticket closed.	Every requirement in the spec is checked against the build, with evidence.
Knowledge	Wikis go stale within a quarter.	Checked against the code on every push. Corrections pinned by people survive rebuilds.
Spotting work	Opportunities sit in reviews and support logs.	Sightings bring them to each seat, cited, one click from a mission.

NOX

Built on Google, from model to metal.

23
GOOGLE PRODUCTS IN USE

Build
where people code

- **Google Antigravity** — where NoX was built; primary `/nox` home
- **Gemini CLI** — `/nox` extension + MCP
- **Jules** — takes a whole build, plan approved by a person

Agents
how NoX thinks

- **Google ADK** — LlmAgent, Workflow graph, sessions, caching, FallbackModel
- **Agent Platform Memory Bank** — team lessons per app and seat
- **Open Knowledge Format** — every knowledge base, portable

Models
right tier for each job

- **Gemini on the Agent Platform** — Fast, Default and Deep tiers · service account, no API key
- **gemini-embedding-2** — hybrid search in pgvector
- **Gemini TTS** — Listen reads replies aloud
- **Gemma 4** — NoX Local, on-device

Run
serverless, scales to zero

- **Cloud Run** — API + web
- **Cloud SQL** — Postgres + pgvector
- **Cloud Storage** — snapshots, captures
- **Secret Manager**
- **Firestore** — tickets
- **Firebase Auth** — Google sign-in
- **Cloud Build**
- **Artifact Registry**

Guard & observe
enterprise-grade

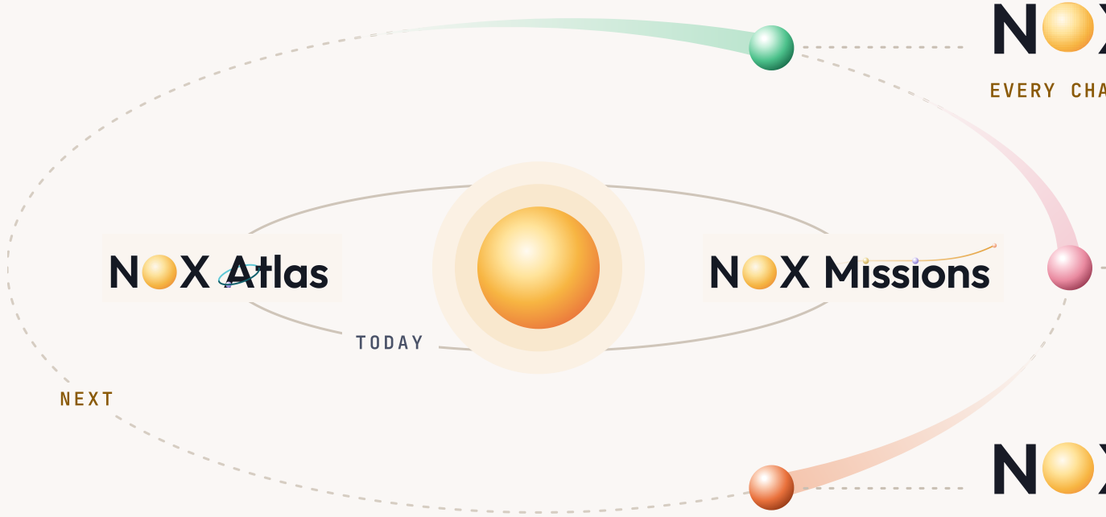
- **Model Armor** — prompt-injection screen
- **Sensitive Data Protection** — pre-commit check
- **BigQuery** — flight recorder
- **Cloud Trace** — per-agent spans
- **Cloud Logging** — usage per job

GEMINI, USED SIX WAYS

Agents with tools · Whole-repo long context · Context caching · Video, voice & images · Embeddings · Speech

Two products today. A bigger NoX, coming soon.

Same sun, same rules. Three new products join Atlas and Missions.



NOX Comms

EVERY CHANGE, EVERYONE INFORMED

Joins Slack, Teams and email. Warns everyone a change touches, gets their acceptance and brings them into the spec.

NOX Orbit

AFTER LAUNCH

Watches shipped changes in production. Tracks the goal, catches failures, finds who owns them and opens the next mission.

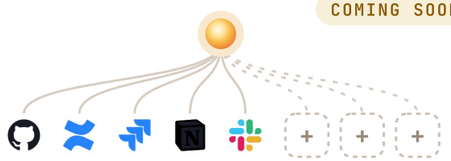
NOX Sunset

CLEAN UP WHAT NOBODY USES

Finds APIs, events and code that nothing uses, and plans their safe removal as missions.

Growing Atlas and Missions

COMING SOON



More connectors

Every source of knowledge, captured.

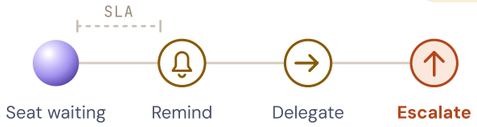
COMING SOON



Roles and team templates

Fits how each team works, with new roles such as Sales.

NEXT



Approval policies and SLAs

Reminders, delegation when someone is out, and escalation when a seat is stuck.

NEXT



Permission-aware knowledge

Each person sees only what they could see in the original tool.

Thank you.

Thank you for reading. We'd love you to try NoX live.

— AstroRangers

- Live app · link
- Film · link
- Code · link